University of Trento

Department of Information Engineering and Computer Science (DISI)

Autonomous Software Agents Project Report

Team autobots

Course: Autonomous Software Agents
Date: 17th June 2026

Authors:
Alessandro Benassi
`alessandro.benassi@studenti.unitn.it`
Student ID: 268241

Filippo Lollato
`filippo.lollato@studenti.unitn.it`
Student ID: 268166

Contents

Introduction

This project is about Deliveroo, a competitive multi-agent game played on a tile grid. Agents move across the map, pick up parcels and carry them to delivery tiles to score points. Parcel rewards decay over time, so the goal is to maximize total delivered score by collecting and dropping off parcels efficiently before they lose value. Extra points can be earned by answering questions and following instructions sent by the Admin in the game chat. The agents of each team cooperate with each other to compete against other teams.

The aim is to develop two cooperating agents:

- Agent 1, BDI physical executor: performs the actual moving, picking, and delivering.
- Agent 2, LLM cognitive coordinator: can carry the same execution as agent one, but needs to have the ability to interpret Admin chat messages, deriving tasks, rules and questions.

PDDL planning is used as an additional tool to solve complex situations.

All the code and the instuctions to run the agents can be found in the GitHub repository:
https://github.com/ale-bena/asa-autobots.git

1.1 Initial state

Both agents connect to the socket via DjsConnect() and they receive the initial state of the game. This last consists in the static grid map, containing width, height and the tiles, which are parsed via an enum:

- WALL
- SPAWN
- DELIVERY
- PAVEMENT
- CRATE_SPAWN, CRATE
- ARROW_UP, ARROW_RIGHT, ARROW_DOWN, ARROW_LEFT

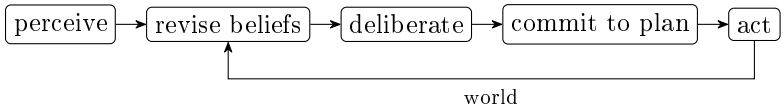
Each agent also recieves their own identity and state (id, name, x, y, score) `via onYou`, stored in beliefs.me. The game parameters, such as observation distance, player capacity, parcel decay interval, movement duration are received separately via onConfig and are kept in beliefs.config and used for calibration.

Before the BDI executor begins acting, the two teammates perform a startup handshake to guarantee both are online. Each agent's execution loop is gated on a `synced` flag: while unsynchronized, it periodically sends a `SYNC_REQ` to its peer; upon receiving one, an agent replies with a `SYNC_ACK` and releases the loop.

BDI Agent

Both agents are built on the same Belief–Desire–Intention reasoning core: each constructs a `BeliefBase` and runs an `IntentionEngine`. The physical executor uses it directly to move, pick up and deliver; the LLM coordinator (Chapter 4) reuses the same structure and extends it with natural-language reasoning.

The agent maintains a model of the world (*beliefs*), evaluates which goals are worth pursuing (*desires*), commits to one (*intention*), and executes it incrementally. A control loop, driven by the game loop, runs each cycle of the schema below at each tick.



2.1 Sensing and Belief Revision

Sensing happens at each movement and whenever something new enters the observation zone. A raw perception frame from the server reports the agent's own state, the visible map configuration, and the nearby agents, crates and parcels, but only within the observation radius, so it is partial and momentary. The agent never reasons on this raw stream directly: every frame is merged into a persistent `BeliefBase` that combines fresh observations with previously acquired knowledge, allowing the agent to reason about entities currently out of sight.

Revision performs three non-trivial inferences:

- **Spatial memory.** Parcels and crates that leave the field of view are retained rather than forgotten.
- **Negative inference.** If a remembered object should be visible, since it falls inside the current radius, but it is absent from the frame, the agent concludes it was collected, decayed, or moved, and prunes it.
- **Decay simulation.** Carried and remembered parcel rewards are aged locally over elapsed time, so value estimates stay accurate between sightings.

The belief base also holds state never delivered by perception: the agent's own identity (`me`), the carried inventory, peers, locked targets, active cooperative contracts, and the `policyRules` issued by the LLM coordinator. A game loop then calls the `IntentionEngine` to reason and act on the current beliefs, while an `onMsg` handler processes Admin prompts (for the LLM agent) and the P2P coordination messages exchanged between the two agents.

2.2 Navigation Graph (Adjacency)

There is no adjacency map sent by the server: the initial state (`onMap`) only provides a flat list of tiles, each with x, y and a type. Adjacency, which is the edges of the navigation graph describing which tile-to-tile moves are legal, is derived on demand from these tile codes by `MapRepresentation.js`.

Two methods implement this. `isAdjacent(from, to)` checks whether a single step is allowed: the target tile must be exactly one step away, must be walkable, and the move must not violate the one-way

arrow gate tiles (a tile cannot be entered against its arrow direction). `getNeighbors(pos)` returns all legal neighbors by applying `isAdjacent` to the four cardinal directions.

Because of the one-way gates, adjacency is directed and not symmetric: a move from A to B may be legal while the reverse is not. For this reason no adjacency table is precomputed; the graph is implicit and evaluated lazily. Pathfinding (`src/mapping/Pathfinding.js`) expands it by repeatedly calling `getNeighbors()` during the search, automatically respecting walls, crates and directional tiles.

2.3 Desires and Goal Selection

Deliberation is throttled to stay stable and avoid redundant pathfinding: goal re-evaluation runs every `GOAL_EVAL_INTERVAL` (12 ticks, about 200ms), but fires immediately when there is no active plan or when the carried inventory changes. When it runs, `selectBestGoal(beliefs, ...)` (`GoalSelector.js`) reads the beliefs and returns a single goal descriptor `{type, targetId, x, y}`, chosen by a strict priority cascade:

1. admin direct commands (`admin_move`, `admin_pickup`, `admin_deliver`);
2. active cooperative contracts with the partner (`rendezvous`, `handoff`, `relay`);
3. an economic, utility-based evaluation over the perceived parcels.

The economic evaluation is the core decision and is *expiration-aware*. For each candidate parcel it estimates the value of the entire trip using A* travel-time estimates, discounting the raw reward by the decay accrued over that time and adjusting it by the coordinator's policy multipliers and bonuses:

$$\text{value} = \big(\text{reward} - r_{\text{decay}} \cdot t_{\text{trip}}\big) \cdot m_{\text{policy}} + b_{\text{policy}}, \qquad \text{utility} = \frac{\text{value}}{t_{\text{trip}}}. \tag{2.1}$$

When already carrying cargo, the agent weighs delivering now against detouring for one more pickup. This trade-off is bounded by three factors: the carrying capacity, the decay risk to the cargo already held, and any required-stack-size incentive, which can make it worth delaying delivery to assemble a larger batch. When no option scores positively, deliberation degrades: it delivers what is held, anchors near a relay tile, patrols the parcel spawn zones, and finally wanders.

2.3.1 Delivery Optimization

Both the utility estimates above and the delivery step itself defer to a dedicated optimizer (`DeliveryOptimizer.js`): on reaching a delivery tile the agent rarely just drops everything, since policy rules may bound which rewards score or require a minimum stack size. `optimizeDeliveryStack` jointly chooses *which subset* of cargo to deliver and *how long to wait* first (candidate wait times are taken from the policy reward boundaries, the instants a parcel crosses an allowed band). Each (subset, wait) pair is scored as

$$\text{score} = R_{\text{delivered}} + \tfrac{1}{2}\, V_{\text{kept}} - \lambda\, t_{\text{wait}}, \tag{2.2}$$

balancing the reward actually delivered against the discounted value of cargo kept back and a waiting penalty; parcels that can never yield positive value are discarded. The same routine is reused during goal selection, keeping valuation and execution consistent.

2.4 Intention Commitment and Plans

The agent is deliberately reluctant to abandon an intention: a newly selected goal replaces the running one only when `shouldPreemptActivePlan(...)` (`PreemptionManager.js`) judges the switch worthwhile, which prevents oscillation between comparable options. The exception is a path blocked by a crate, which suspends the active plan and injects a PDDL `push_crate` plan instead (Chapter 3).

On commit, `instantiatePlanRecipe(goal)` maps the goal type to a generator-based plan recipe. These generators yield one primitive step at a time and can be suspended and resumed via a suspension

stack, so a priority plan can interrupt a delivery and the delivery resumes afterwards. Suspended plans that have gone stale, since their target disappeared or became obsolete, are discarded rather than resumed.

2.5 Execution

Each cycle advances the active plan by one step: (`move`, `pickup`, `putdown`, `wait`, `say`), handed to `dispatchAction` (`ActionDispatcher.js`) which emits it on the socket. The plan is told whether the previous step succeeded, so it can react to failures rather than continue blindly.

Two behaviours sit outside the normal deliberation cycle. The first is an *opportunistic pickup*: if the agent finds itself standing on a free parcel and is still below its carrying capacity, it grabs the parcel immediately, before any goal evaluation. This is done to avoid leaving reachable parcels on the floor and wasting potential value.

The second is *collision avoidance* between the two teammates, designed so they never deadlock over a tile during testing. Before moving, each agent uses the partner's broadcast position and planned next step to detect a possible conflict where both aim for the same tile (see Section 5.1). When this happens one agent must yield, and this is decided deterministically by comparing agent identifiers, so both reach the same conclusion independently without any negotiation(higher ID yields). The yielding agent pauses 150 ms to let the other pass. If a move is still rejected, the agent waits one tick (100 ms) and retries, and after two repeated collisions on the same step it marks that tile as blocked and re-plans a route around it. The block is cleared after a short timeout so the tile stays reusable. When the two agents repeatedly block each other, a `stuckCounter` escalates the response: it first waits a tick, then steps aside to a free neighbouring tile and re-plans, and as a last resort abandons the plan so deliberation can start fresh. When a plan finishes, the engine resumes any suspended plan or clears the goal and re-deliberates on the next tick.

PDDL Use

The BDI loop is complemented by a symbolic planning layer reserved for one sub-problem: clearing a pushable crate that blocks a corridor. PDDL (Planning Domain Definition Language) is an exact symbolic planner, useful for planning out a sequence where moving crate is involved.

3.1 Role and scope

Ordinary tile-by-tile navigation stays with A*, which is fast and runs constantly; a general-purpose solver called on every query would be far too slow for the control loop, so PDDL is kept off the main loop while still guaranteeing a correct plan where local heuristics fail.

The planner is reached through the `pddl-client` interface. To maximize performance we decided to locally host the PDDL solver on `http://localhost:5001`.

3.2 The domain

The domain is comprised of two families of actions:

- **Movement** (`move-right/left/up/down`): the agent moves to an adjacent tile, provided that it's marked `clear`, meaning it is currently known to be free of crates and other agents.
- **Pushing** (`push-right/left/up/down`): when the agent, a crate, and a destination tile are

aligned in the same direction, the agent advances onto the crate's tile and push the crate one step forward in a clear and allowed direction(crate tile, crate-move-capable flag). Clear predicates are adjusted accordingly.

Directionality is encoded as explicit adjacency predicates (right/left/up/down) rather than coordinates, so the planner reasons over a symbolic graph.

3.3 The problem

Each time a crate is blocking a path, a fresh problem is compiled from the agent's current beliefs. The compiler scans the walkable tiles and produces four kinds of facts:

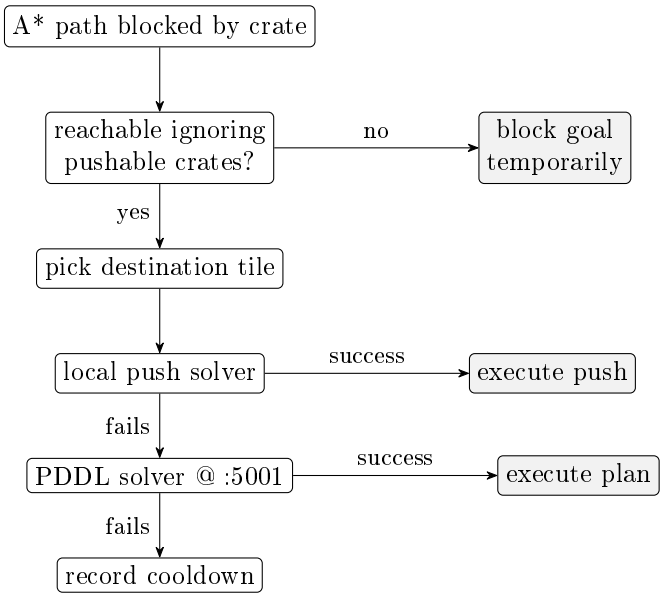
- **Tiles and connectivity:** one object per walkable tile, together with the directional adjacency facts, obtained from the same move-legality check the agent uses for ordinary navigation.
- **Crate-capable tiles:** a `crate-move-capable` flag on the tiles that can physically hold a crate (the crate-spawn and crate-move types).
- **Free tiles:** a `clear` flag on every tile not occupied by a crate or a peer agent (both treated as obstacles).
- **Objects:** the agent's position, the target crate, and every other crate, each as a distinct object.

Only *physical* obstacles mark a tile as non-clear, *Transient* goal blocks, such as a delivery zone that is momentarily unreachable, are left out, because including them would label otherwise reachable tiles as blocked and could lead the planner to wrongly conclude that no solution exists.

The goal is a single fact: the blocking crate is relocated onto a chosen clear tile, that is, an empty crate-capable tile. The plan the solver returns, a sequence of moves and pushes, is then translated back into grid steps and handed to the executor as a plan.

3.4 Crates and strategy

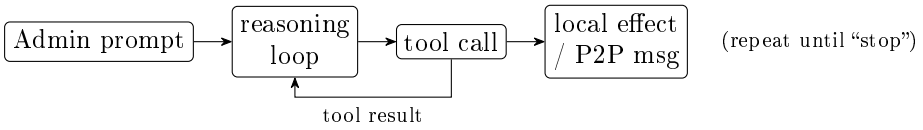
Crates are a dynamic, *relocatable* obstacle in the Deliveroo game, which is why they receive dedicated planning. The agent first picks a sensible destination for the blocking crate: a nearby clear, crate-capable tile, reachable via a push whose "push-from" side the agent can actually stand on. Resolution then proceeds in tiers, cheapest first:



To manouvre around multiple crates we rely on the PDDL solver, which can chain multiple pushes and moves. Failed solves are recorded with a 1 s cooldown so the agent does not repeatedly hammer the planner on the same blockage. A crate whose solve keeps failing is then treated as a wall.

LLM Agent

This second agent is a cognitive coordinator. It is built on the same BDI core as the executor (Chapter 2), so it too can move, pick up, deliver, and clear crates with PDDL (Chapter 3). What sets it apart is an additional natural-language reasoning layer: it receives instructions from the Admin in the chat, reasons about them, and translates them into concrete commands and policies for the whole team. It never acts on this reasoning directly; every effect is mediated by a tool or a peer-to-peer message and ultimately carried out by a BDI agent. Of the three candidate models available, the one selected is `llama-3.3-70b-lmstudio`.



4.1 The reasoning loop

A prompt triggers an iterative, single-step reasoning loop driven by a language model under a fixed system prompt. The model returns one validated JSON object per turn, of exactly one of three kinds:

- **tool**, a request to execute one of the available tools;
- **answer**, a textual output to return;
- **stop**, a signal that the prompt has been fully handled.

Chain-of-thought is elicited through delimited reasoning that is stripped before parsing, and the model runs at zero temperature for determinism. After each turn the model executes the requested tool (or records the answer), feeds the result back as the next input, and repeats until a `stop` is produced. Outputs that violate the schema are rejected and re-requested with an error description, up to three attempts. The loop advances one step at a time by design, so a message containing several tasks is decomposed and resolved alongside intermediate answers (Section 6.0.1).

4.2 Prompt construction

The system prompt is fixed for the whole run and organised into clearly delimited sections. A *role* description frames the model as the team's reasoning brain and lists the agent identifiers. A *definitions* block gives precise meanings for the three concepts the coordinator must tell apart: a *reward* (the points gained or lost by carrying out an instruction, possibly written as an expression), a *task* (a single unit of work, where one message may contain several), and a *rule* (a standing instruction that changes how future tasks are scored). A set of operating *rules* then fixes the procedure the model must follow, and a *response-format* contract pins down the exact JSON it may emit. The prompt ends with the available tools and a bank of worked examples.

The definitions and operating rules are what make the behaviour reliable rather than improvised. Because *reward*, *task*, and *rule* are defined explicitly, the model can consistently separate a one-off rewarded task from a standing rule, the distinction that drives feasibility gating. The operating rules encode the rest of the discipline: expressions must be solved with the arithmetic tool instead of

computed by the model, multi-task prompts must be decomposed and answered in presentation order, and unknown facts such as positions or map extremes must be fetched with a context tool rather than guessed. They also carry targeted guardrails learned from failures, for instance checking that a target tile is walkable before a move is ordered, and never sending both agents to the same tile.

Beyond stating the rules, the prompt teaches them largely by example. It is few-shot, with curated cases covering feasibility checks, multi-task decomposition, rule application, holds and resumes, and cooperation contracts, each showing the full turn-by-turn exchange. Finally, the tools section of the prompt is generated automatically from the same registry the coordinator executes against, so the descriptions the model reads can never drift from the tools that actually run.

4.3 Message filtering and history

Two filters precede the reasoning loop, both enforced in code rather than left to the prompt. First, only messages originating from the Admin are treated as candidate instructions; messages from any other agent are routed to the P2P handler instead. Second, the coordinator distinguishes actionable instructions from Q&A messages carrying no reward: a rewardless, non-command message produces a `stop` with no chat output at all. Conversation history is kept in full but passed to the model only when needed and in filtered form, namely past questions and answers only, never the intermediate reasoning or tool calls, so the prompt stays compact.

4.4 Feasibility gating

The central reasoning discipline is that effectful actions are gated on a feasibility check. Any expression, whether a reward value or a logical condition, must be resolved through the arithmetic tool rather than computed by the model itself. World- or rule-changing actions (movement, variable setting, cooperation proposals) are reward-gated: they are issued only once the arithmetic evaluation has confirmed a strictly positive reward for the current prompt. Once confirmed, the gate holds for the rest of that prompt, so later numeric evaluations such as computing coordinates do not re-lock it.

Two classes are intentionally exempt. *Standing policy rules* ("from now on…", penalties, multipliers) are announcements of an effect, not rewarded tasks, so they are always applied regardless of reward, since they describe how the team adapts to penalties. *Control and query* actions (hold, resume, context and history lookup, contract cancellation) are likewise exempt, as they carry no reward of their own.

4.5 Tools

Tools fall into two groups. *Read* tools retrieve facts without side effects: arithmetic evaluation, the local context snapshot (own and peer state, parcels, map extremes), and conversation history. *Act* tools change behaviour: directing an agent to a coordinate, applying scoring or avoidance policy rules, setting shared variables, holding or resuming agents, and proposing cooperation contracts.

The key design point is symmetry of effect. When an act tool targets the coordinator itself (or "all"), it mutates the coordinator's own beliefs directly; when it targets the executor (or "all"), it additionally emits a peer-to-peer message so the executor applies the same change to its belief base. The same vocabulary governs both agents through one uniform mechanism, detailed in Chapter 5.

Coordination and Messages

The two agents form a team, and all of their cooperation flows through the game's chat channel as structured messages. This chapter describes the transport, the message vocabulary, and the protocols built on top of it. The same machinery serves two roles: the LLM coordinator uses it to push commands

and policies to the executor, and both agents use it to continuously share state and resolve conflicts.

5.1 Agent Communication

Messages are exchanged over two primitives: `emitSay` delivers a message privately to a single recipient, while `emitShout` broadcasts to everyone. The team prefers the private channel, a directed `emitSay` to the partner, and falls back to a shout only if the direct send fails. This keeps the public chat uncluttered and avoids leaking coordination to competing teams. Incoming chat is pre-filtered: each agent ignores its own messages and anything that does not match the cooperative JSON schema, so ordinary Admin text and opponent chatter never reach the protocol handlers.

Underlying everything is a steady stream of `PEER_STATUS` heartbeats sent directly to the partner a few times per second. Each carries the sender's position, score, carried inventory, planned path and next step, and known crates. This keeps each agent's model of its partner current even when it is out of sensing range, shares crate sightings so both navigate around the same obstacles, and lets each agent see in advance when they are converging on the same tile, so one can yield or re-deliberate toward a different target without any explicit negotiation or locking.

5.2 Message Structure

Every cooperative message is a JSON object with a `type` field that selects a handler; the remaining fields are the payload. The vocabulary groups into three families:

- **Status:** `PEER_STATUS`, for position, cargo and obstacle sharing, and the `SYNC_REQ`/`SYNC_ACK` pair used once at startup so neither agent acts before its partner is online (Section 1.1).
- **Direct commands** (coordinator to executor): `MOVE_TO`, `HOLD`, `RESUME`, `APPLY_RULES`, `SET_VARIABLE`, and `INSTRUCT_SAY`. Of these, only `MOVE_TO` expects a reply: the executor returns a `MOVE_TO_ACK` once it has reached the target or failed to, so the coordinator knows the move is complete. `INSTRUCT_SAY` is the coordinator asking the executor to relay the LLM's chat answer to the Admin, a redundant delivery path that makes the reply more likely to reach the Admin.
- **Contracts:** `PROPOSE_CONTRACT`, `ACCEPT_CONTRACT`, `CLOSE_CONTRACT` for the propose/accept/close lifecycle, plus `SIGNAL_READY` and `RELEASE_CARGO` as the main status transitions: `SIGNAL_READY` announces that an agent has dropped its cargo and stepped aside so the partner may collect it, and `RELEASE_CARGO` marks the cargo as released for the partner to take.

On receipt, a handler applies the message directly to the agent's own belief base. This is the mechanism by which a coordinator command takes effect remotely: a `MOVE_TO` becomes a high-priority `admin_move` contract (and implicitly releases any standing hold), an `APPLY_RULES` updates the policy set, `HOLD`/`RESUME` toggle the paused state, and a `SET_VARIABLE` writes shared memory. The executor thus needs no special command interpreter, since coordination and perception update the same beliefs through the same revision path.

5.3 Coordination Workflow

Contract handshake. Multi-agent manoeuvres are negotiated as *contracts* rather than hard-coded. One agent proposes, the other validates that the target tile is valid and walkable and accepts, both mark it active, and either may cancel it later. We chose a propose/accept handshake over simply commanding the partner so that a contract is never acted upon unless both agents agree it is feasible, which avoids one agent waiting forever for a partner that cannot reach the meeting point. Three cooperation patterns are built on this handshake.

- **Meet-and-wait** (`RENDEZVOUS`, and `CLEARING` as a special case): both agents converge within a

radius of a tile and pause. We decided to model "clear this gate" as a rendezvous rather than a separate mechanism, because mechanically it is the same act, bring an agent to a point and hold.

- **Hand-off** (`HANDOFF`): one agent carries its cargo to a shared tile, drops it, and steps aside so the other can collect it. This exists for the case where the carrier cannot reach the delivery zone but its partner can.
- **Relay** (`RELAY`): a persistent contract in which a designated courier repeatedly collects parcels and brings them to the partner which has the duty to deliver them.

Blocking commands wait for confirmation before the coordinator proceeds: a directed move resolves only once a `MOVE_TO_ACK` or a position match is observed, a deliberate choice so the LLM's reasoning never advances on an unconfirmed physical effect. Control actions such as `RESUME` clear outstanding contracts so no agent is left waiting in a coordination loop.

Tests

Both the agent have been tested locally during the building process continously, to find bugs, test different configurations, and improve them.

Extra attention was dedicated on checking the structure of the messages and actions proposed by the LLM, since we cannot blindly rely on a non-deterministic way of extracting intentions from the Admin prompt.

To facilitate testing on multiple scenarios we decided to build a flexible map-builder tool (`run_map_builder.js`, served at `localhost:3000` through `npm start`). After manual test, we implemented automated unit and coverage tests, which guarantees that the current behaviour remains consistent. This is done via the built-in node test system, and code actions to verify that new code changes comply with the tests.

6.0.1 Considerations

We have found creating an appropriate, logically sound, system prompt for the LLM not trivia, as it frequently tried to vier off the guardrails.

Some known limitations we found were: the occasional inability for the LLM to reason about associating multiple rewards to different tasks, resulting in missed opportunities and unhandled requests; not performing the actions in the same sequential order as they were provided, leading to out of order answers.

Conclusion

With this project we were able to achieve Summarize the main contributions and discuss future developments.

Bibliography

[1] AI-Planning. Planning as a service. https://github.com/AI-Planning/planning-as-a-service, 2026.

[2] unitn-ASA. Deliverooagent.js. https://github.com/unitn-ASA/DeliverooAgent.js, 2026.

[3] unitn-ASA. Deliveroo.js. https://github.com/unitn-ASA/Deliveroo.js, 2026.